

HIGH PERFORMANCE COMPUTING FOR SCIENCE AND ENGINEERING

Harvard University
CS 2050

Lecture 5
February 10, 2026



2025-2026 Spring

[Home](#)

[Announcements](#)

[Course Calendar](#)

[Q & A](#)

[Zoom](#)

[DCE Class
Recordings](#)

[Quizzes](#)

[Gradescope](#)

[People](#)

[Grades](#)

Quiz 2

Due Feb 10 at 11:59pm

Points 10

Questions 10

Available Feb 9 at 12am - Feb 10 at 11:59pm

Time Limit 60 Minutes

Instructions

The quizzes are designed to take no more than 30 minutes, but one hour will be allowed.

Do not click into the quiz until you are ready to take it.

[Take the Quiz](#)

SPACK

- Load the CS-2050 environment

```
[$] . ~/161588/spack/share/spack/setup-env.sh  
[$] spack env activate -p CS-2050
```

homework-1

Internal template

Edit Pins

Watch 0

Fork 0

Star 0

Use this template

main



Go to file



Code

About



No description, website, or topics provided.

Readme

Activity

Custom properties

0 stars

0 watching

0 forks

Releases

No releases published

[Create a new release](#)

	laz380 and Chuck Witt	public release.	0aab4ce · yesterday	1 Commits
	question-1	public release.		yesterday
	question-2	public release.		yesterday
	question-3	public release.		yesterday
	question-4	public release.		yesterday
	question-5	public release.		yesterday
	question-6	public release.		yesterday
	question-7	public release.		yesterday
	README.md	public release.		yesterday



Gradescope Submission #20

Open

ryj579 opened this issue 2 days ago · 1 comment



ryj579 commented 2 days ago



Hello,

The submission instructions on the repo say to create a text file with the relevant information, but Gradescope only accepts either images or PDFs. Is it expected for us to create a word document or something with the name, hash, and link, or is there another way to submit to Gradescope I'm missing?

Thanks!



1



wcw398 commented 2 days ago • edited



Thanks - it looks like you are correct and the assignment type I selected in Gradescope is more restrictive than I realized. Gradescope also has a slightly different assignment type that accepts any file - I will use that for Homework 2 and beyond.

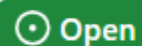
For Homework 1, please submit a PDF (e.g. use Word and print to PDF).

Apologies for the confusion.



1

SSH to the cluster #12



Open

chb1856 opened this issue 5 days ago · 4 comments



chb1856 commented 5 days ago



Hello. Setting up SSH from my machine to the cluster. I have the key downloaded, but cannot ping that host. Do I need to be on a VPN? And which hostname would be correct, so I am on a host that is safe to run jobs on?

Thanks!



wcw398 commented 5 days ago



Hello - I'd recommend holding off for now. The early experiments surfaced some issues we need to address with the cluster admins. We'll circulate some more detailed instructions if those issues can be resolved.



1

PROJECTS

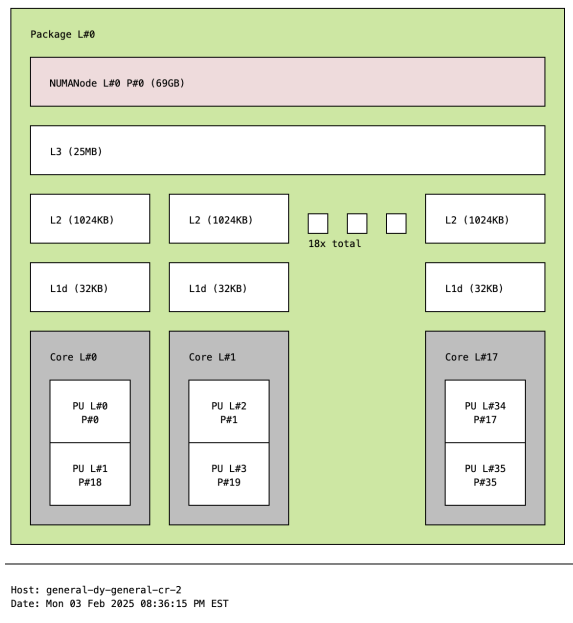
- We will release full project specification in Week 6
- Standard project:
 - Individual
 - Choose an algorithm and parallelize it using OpenMP, MPI, and CUDA
 - Deliverable will be brief report in a blog post style
- Some possibility of separate (optional) track that would allow students to propose something different for their project
 - Difficult to do consistently/fairly
 - Open to input on this idea

WHAT IS SHARED MEMORY PARALLELISM

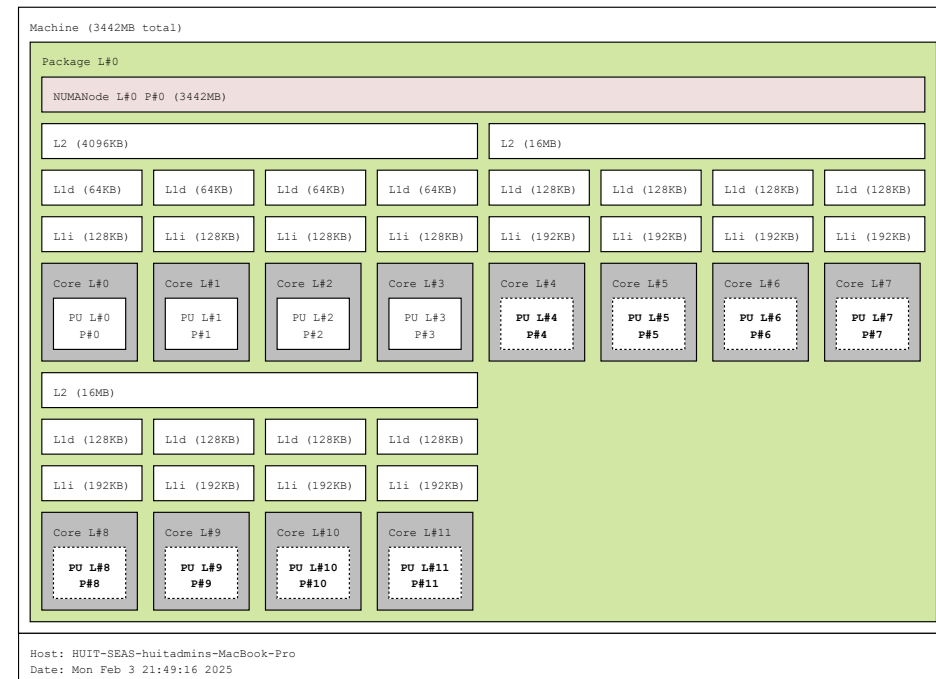
- We have seen that a reinterpreted Moore's Law applies in the multicore era
→ CPU core count per chip doubles roughly every two years, while individual cores remain constrained by power limits
- However, we have not talked about *how* cores cooperate with each other
 - When cores cooperate they need to share and/or exchange information
 - One approach is to share a memory space where multiple cores have *read* and *write* privileges.
- *Assume you have two tasks running with shared memory:*
 - Is it safe if both tasks read from the same memory location?
 - Is it safe if one task reads from and the other writes to the same memory location?
 - Is it safe if both tasks write to the same memory location?

MULTICORE ARCHITECTURE

Example multicore architectures



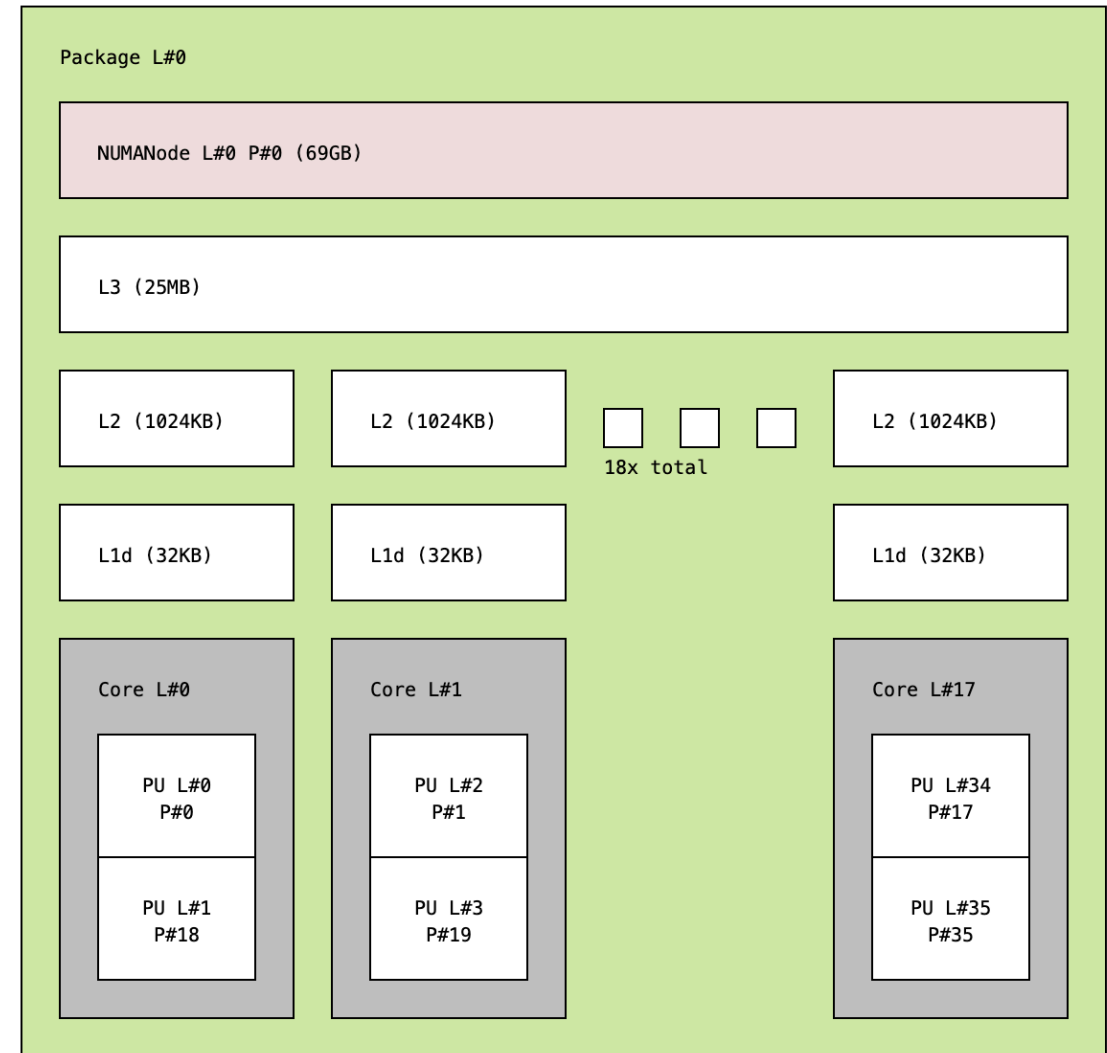
Multicore architecture similar to the nodes on the CS 2050 cluster



Apple silicon. Different vendors produce slightly different architectures.

PARALLELISM IN HARDWARE

- Multiple physical cores (obviously)
- Multiple physical memory levels → caches can be shared among cores or be exclusive
- Various lower level architectural details like pipelining



PARALLELISM IN APPLICATIONS

```
int a[1024];

int main(void)
{
    // task 1: set first half of the array to 1
    for (int i = 0; i < 512; ++i) {
        a[i] = 1;
    }
    // task 2: set second half of the array to 2
    for (int i = 512; i < 1024; ++i) {
        a[i] = 2;
    }
    return 0;
}
```

Two forms of parallelism:

Data-level parallelism

Arises when there are many data items that can be operated on at the same time.
→ Vectorization and GPUs.

Task-level parallelism

Arises when different tasks can be executed in parallel.
→ Shared memory can be an asset when threads need to interact.

PROCESSES AND THREADS

Processes:

- A process is an execution sequence within the operating system, a *running* program with a process ID
- Own a *virtual address space*
- Have an *execution state* (program counter, register values)
- Are managed by the OS

Software threads:

- Can execute a sequence of instructions within a running process
- All threads associated to the same process share the application data in main memory
- Threads are cheaper to create than processes
- The programmer can control the creation of threads

PROCESSES

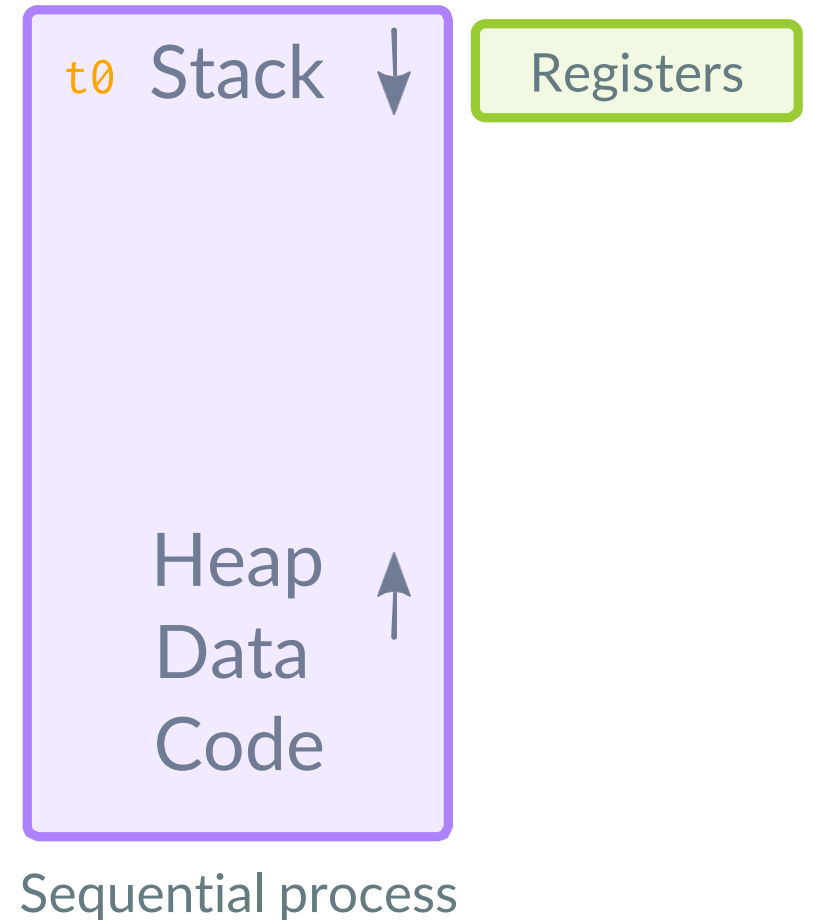
- No process can access another's virtual memory (and therefore physical memory)
→ a basic safety requirement in multi-user environments such as Linux
- Communication between processes is important when they cooperate on a problem (e.g. MPI)
- Inter-process communication is implemented by the OS with several mechanisms: signals, files, pipes, sockets or shared memory



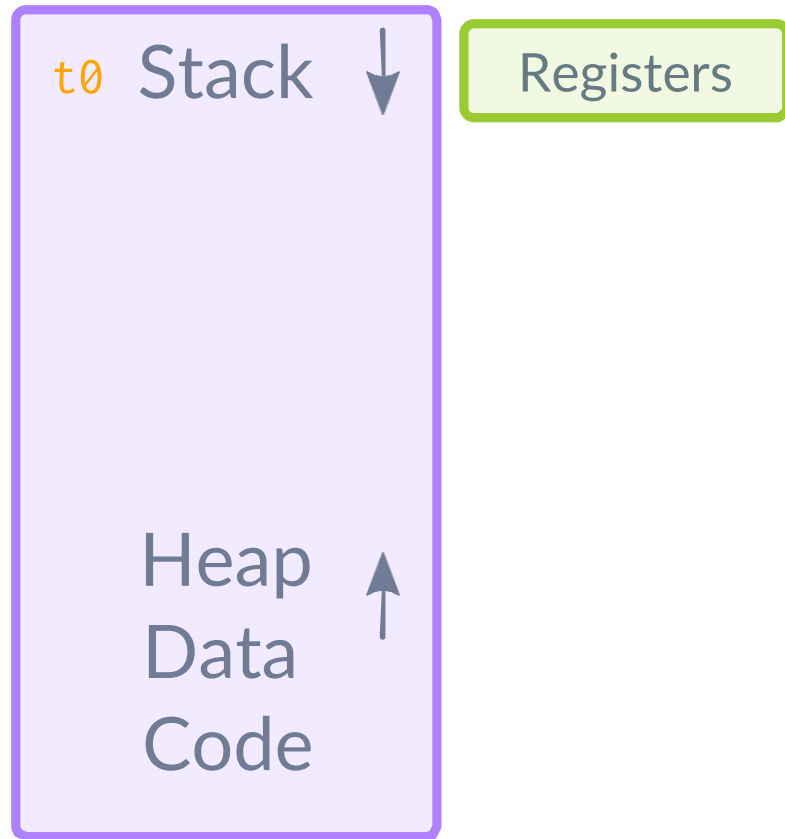
Simplified representation of the virtual memory space of a Linux process.
The stack grows to lower memory addresses, while the heap to higher ones.

SOFTWARE THREADS

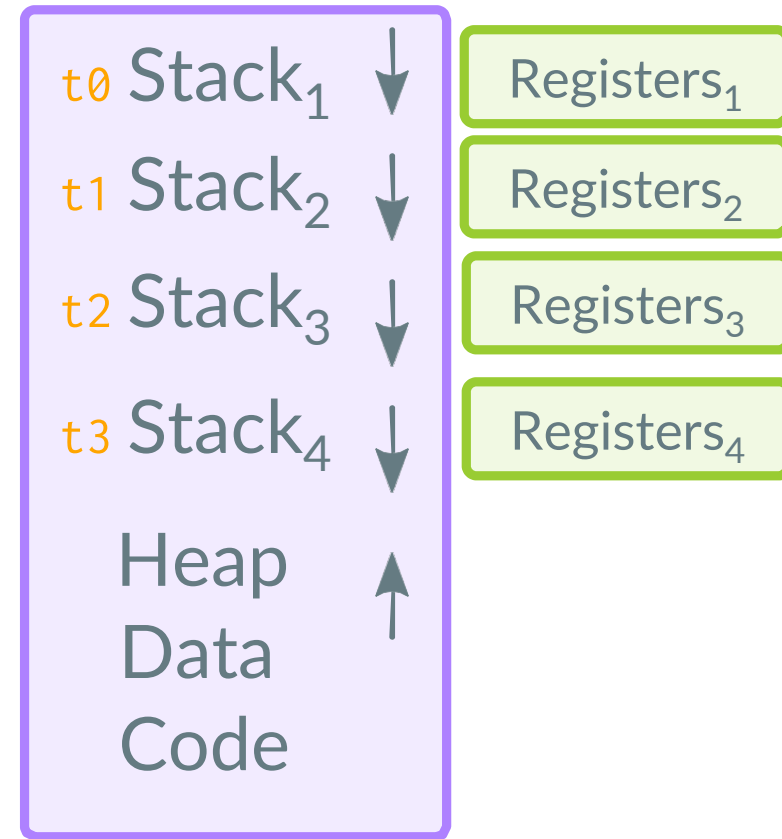
- An independent stream of instructions scheduled to run by the OS (runs on a CPU core or possibly hardware thread)
- *A process can create many threads* (more than there are physical cores)
- Each thread:
 - *shares the memory* of the process they belong to with other threads
 - has its own state and some private memory
 - is free to run on *any* available processor (core or hardware thread)
 - *is cheap to create*



SOFTWARE THREADS



Sequential process



Multithreaded process (concurrent execution).
Communication between threads t_0 , t_1 , t_2 and t_3 is through *shared main memory*

A SIMPLE CLASS FOR COUNTING

```
#include <iostream>

class Counter
{
public:
    Counter() : count_(0) {}

    void increment() { ++count_; }
    unsigned int get_count() const
    {
        return count_;
    }
private:
    unsigned int count_;
};

int main(void)
{
    Counter counter;
    for (int i = 0; i < 100; ++i) {
        counter.increment();
    }
    std::cout << counter.get_count() << '\n';
    return 0;
}
```

What the code does:

- The Counter class defines increment and get_count in the public interface
- Any new instance of Counter is initialized to 0
- Incrementing the counter is done with the ++ prefix increment operator
- The main function increments the counter 100 times and prints the final count to stdout

A SIMPLE CLASS FOR COUNTING

```
#include <iostream>

class Counter
{
public:
    Counter() : count_(0) {}

    void increment() { ++count_; }
    unsigned int get_count() const
    {
        return count_;
    }
private:
    unsigned int count_;
};

int main(void)
{
    Counter counter;
    for (int i = 0; i < 100; ++i) {
        counter.increment();
    }
    std::cout << counter.get_count() << '\n';
    return 0;
}
```

Sequential execution:

- A single thread (called the primary thread) calls the `increment()` method arbitrarily to increment the counter
- The code executes sequentially and the instructions in the compiled binary execute in the **correct (program) order**.
- We expect the program to print 100 every time we call it:

```
$ g++ -o counter counter.cpp
$ ./counter
100
$ ./counter
100
$ ./counter
100
$ .counter
100
```

A SIMPLE CLASS FOR COUNTING

```
#include <iostream>

class Counter
{
public:
    Counter() : count_(0) {}

    void increment() { ++count_; }
    unsigned int get_count() const
    {
        return count_;
    }
private:
    unsigned int count_;
};

int main(void)
{
    Counter counter;
    #pragma omp parallel for
    for (int i = 0; i < 100; ++i) {
        counter.increment();
    }
    std::cout << counter.get_count() << '\n';
    return 0;
}
```

Parallelize the code:

- We can easily parallelize the code by adding a `#pragma` preprocessor macro for OpenMP (more soon)

Multithreaded execution:

- Now multiple threads in a team may call the `increment()` method in the loop body at arbitrary time *in parallel*
- The single Counter instance `counter` is *shared* by all threads in the team (hence "shared memory")

What memory operations can threads perform on the shared counter variable?

A SIMPLE CLASS FOR COUNTING

```
#include <iostream>

class Counter
{
public:
    Counter() : count_(0) {}

    void increment() { ++count_; }
    unsigned int get_count() const
    {
        return count_;
    }
private:
    unsigned int count_;
};

int main(void)
{
    Counter counter;
    #pragma omp parallel for
    for (int i = 0; i < 100; ++i) {
        counter.increment();
    }
    std::cout << counter.get_count() << '\n';
    return 0;
}
```

Multithreaded execution:

- Now multiple threads in a team may call the `increment()` method in the loop body at arbitrary time *in parallel*
- The single Counter instance `counter` is *shared* by all threads in the team (hence "shared memory")
- Compile the code, run with 4 threads:

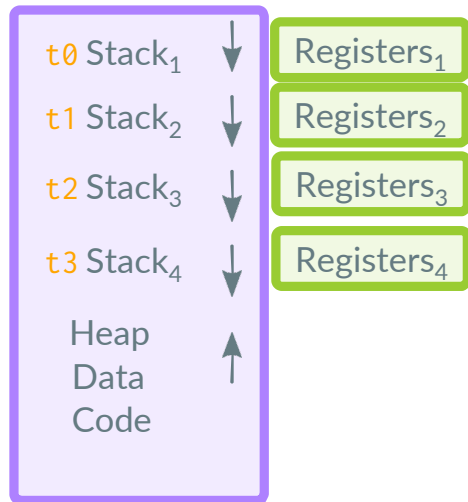
```
$ g++ -fopenmp -o counter counter.cpp
$ OMP_NUM_THREADS=4 ./counter
87
$ OMP_NUM_THREADS=4 ./counter
65
$ OMP_NUM_THREADS=4 ./counter
94
$ OMP_NUM_THREADS=4 ./counter
91
```

Why is the result inconsistent and different from 100?

RACE CONDITIONS

- From sequential programming, you are accustomed to assume that your program will take a particular trajectory through the execution state space to yield a correct result (program order)
- In shared memory programming multiple such trajectories are possible, ***all of them must yield the correct result***
- The problem in the previous example is a ***race condition*** on the variable `count_` in shared memory. If multiple threads *read* from this variable everything is fine (`get_count()` is *thread-safe*) but if multiple threads *write* to this variable we encounter inconsistent and wrong results. We write to `count_` in the `increment()` method which is not *thread-safe*.

RACE CONDITION IN COUNTER CLASS



Each thread maintains its own *state* of registers and program counter.

The `increment()` method in the `Counter` class result to memory.

```
void increment() { ++count_; }
```

When we disassemble this code and inspect the machine instructions that are executed for `++count_` we find essentially *3 instructions*:

```
mov edx, DWORD PTR [rax] ; load value at address rax into register edx (count_)
add edx, 1 ; add 1 to value in register edx, store result in edx
mov DWORD PTR [rax], edx ; write the value in edx to memory location rax
```

The data in registers `rax` and `edx` is *private to each thread*. If two threads execute this code, a **race condition** might look like this:

```
mov edx, DWORD PTR [rax] ; thread 0 loads current value of count_
mov edx, DWORD PTR [rax] ; thread 1 loads current value of count_
add edx, 1 ; thread 1 increments count_ and stores result in edx
add edx, 1 ; thread 0 increments count_ and stores result in edx
mov DWORD PTR [rax], edx ; thread 0 writes register edx to memory
mov DWORD PTR [rax], edx ; thread 1 writes register edx to memory (same value!)
```

ATOMIC OPERATIONS

- Note that the increment `++count_` is composed of 3 *instructions* at the machine level
- What appears to us as a *single line of source code* is effectively "3 lines" of machine code
- This is the root of race conditions in multithreaded execution flow. Be aware that race conditions are ***hard to debug*** and one of the reasons why shared memory programming is more difficult than sequential programming.
- The statement `++count_;` ***is not atomic!***
- An atomic operation is an operation that will always be executed without any other thread being able to read or change state that is read or changed during the operation. It is effectively executed as a single atomic step (smallest building block).
- A correct (*thread-safe*) `increment()` method is achievable with an atomic pragma

```
void increment()
{
    #pragma omp atomic
    ++count_; // now this line of code is atomic (and thread-safe)
}
```

CRITICAL SECTION

```
#include <iostream>

int main(void)
{
    Counter counter;
    #pragma omp parallel for
    for (int i = 0; i < 100; ++i) {
        counter.increment(); // critical section
    }
    std::cout << counter.get_count() << '\n';
    return 0;
}
```

- Line 8 in the code on the left is a **critical section** because multiple threads try to modify *the same memory location*
- Dangerous because the relaxed memory consistency model allows for *race conditions* without our supervision.

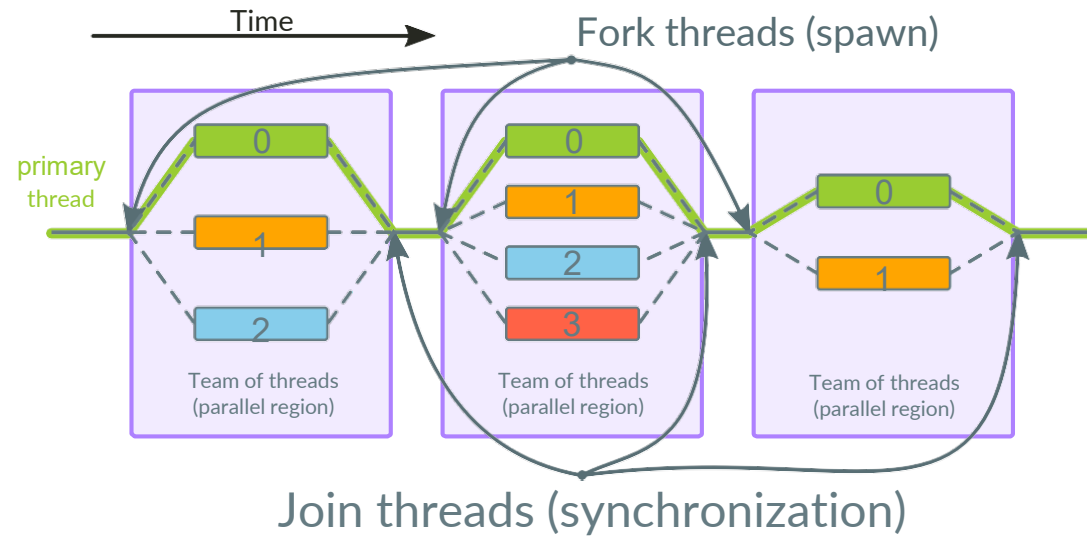
Atomic primitive sufficient here, but other techniques better for more complex cases.

```
class Counter
{
public:
    Counter() : count_(0) {}

    void increment() {
        #pragma omp atomic
        ++count_;
    }

private:
    unsigned int count_;
};
```

CREATION OF THREADS — FORK/JOIN MODEL



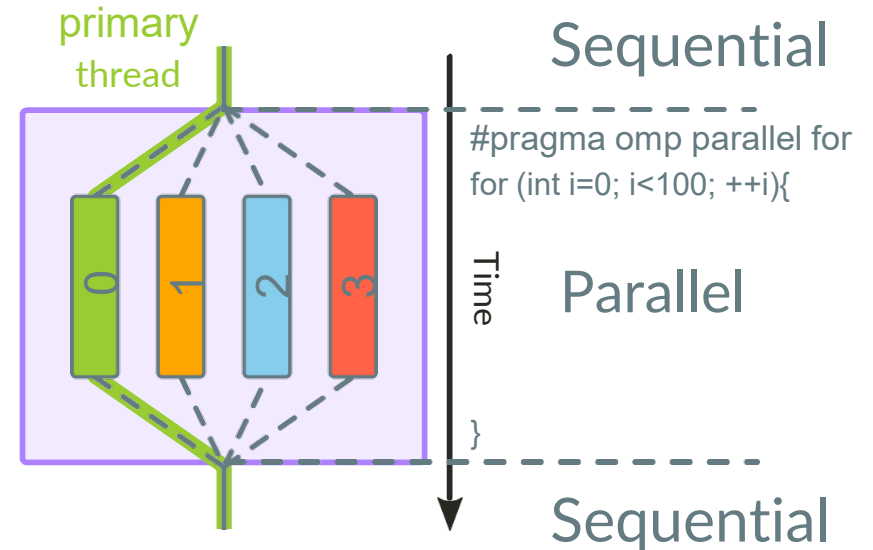
The longer a team of threads can execute in parallel, the higher the *parallel fraction* of your code

- The primary thread is the main thread that enters the main function in your program. This thread also exists if you execute sequentially.
- It controls the *creation of child threads* with a fork/join operation
- Child threads within a *team of threads (parallel region)* may create sub-teams of threads (nested parallelism)
- Threads are *joined* together at *synchronization* points (barrier, memory fence)
- After synchronization, *only* the primary thread continues execution

CREATION OF THREADS — FORK/JOIN MODEL

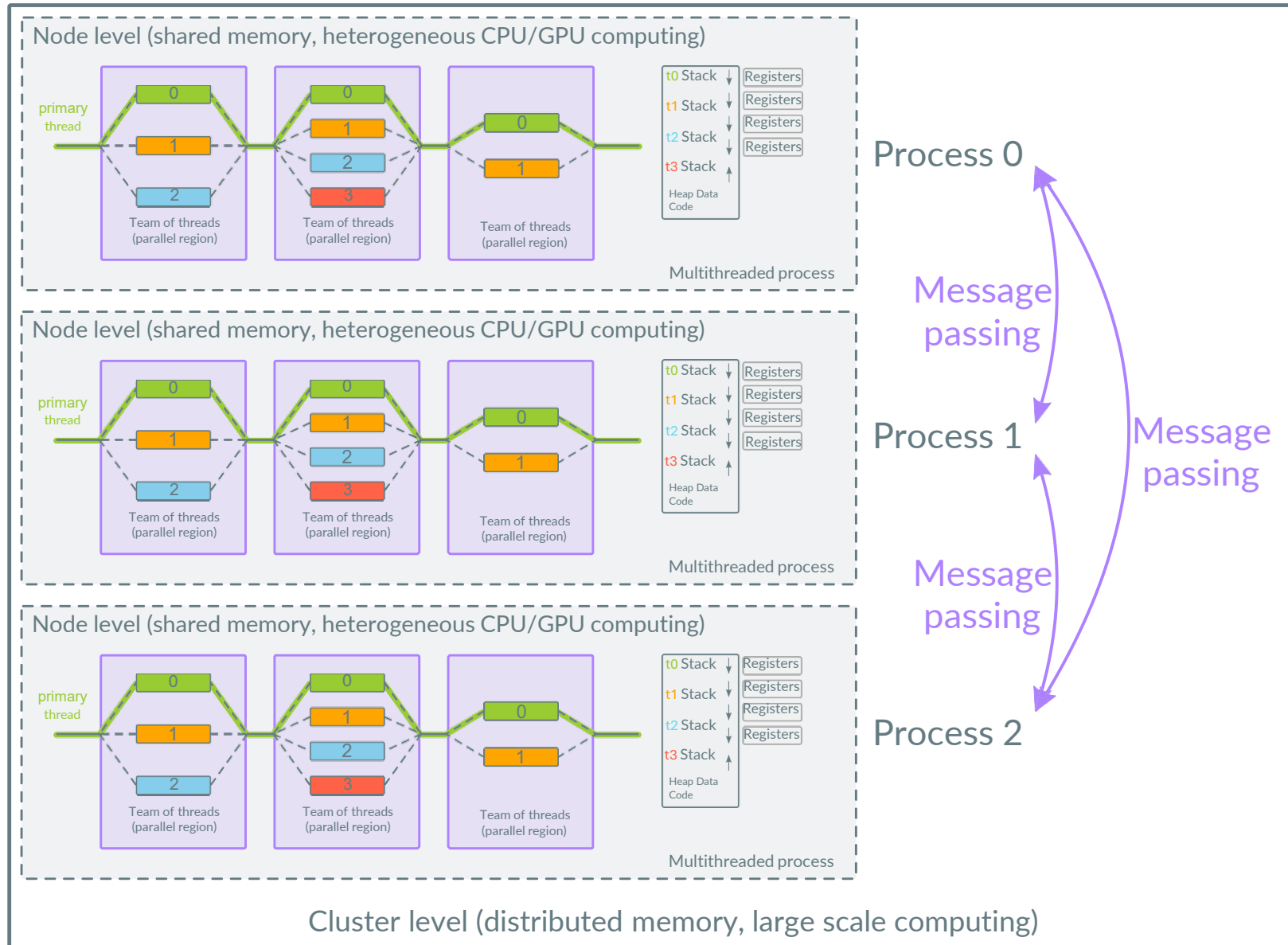
The Counter example from before:

```
#include <iostream>
int main(void)
{
    Counter counter;
    #pragma omp parallel for
    for (int i = 0; i < 100; ++i) {
        counter.increment();
    }
    std::cout << counter.get_count() << '\n';
    return 0;
}
```



- The highlighted code *spawns* a parallel region using the OpenMP directive `#pragma omp parallel`
- The body of the loop is executed in parallel where the work contained in the 100 iterations is divided among the threads (*trying to exploit task-level parallelism*)
- At the end of the loop there is an *implicit synchronization*, the threads are joined together again and the primary thread continues sequential execution

WHERE WE ARE GOING (THE BIG PICTURE)



RECAP SO FAR

- Shared memory programming belongs to the MIMD category of parallel computers. We can exploit data-level parallelism and/or task-level parallelism using shared memory
- Software threads (often we just call them "threads") are lightweight logical entities that have their own state (register values, program counter) and can run on any hardware core (scheduled by the OS)
- A race condition happens when two (or more) threads try to *write* to the same memory location at the same time. The program behavior is not deterministic if you have race conditions (why?)
- Threads are forked (or spawned) into teams of threads that operate in *parallel regions*. The longer they are spawned, the larger the available parallelism in your application. Threads are joined at *synchronization points*
- We employ shared memory on the *node-level*
- Multithreaded programs in a shared memory architecture require constraints on the read/write instruction order to ensure correct parallel programs
- **You** must enforce these restrictions to ensure correct parallel programs

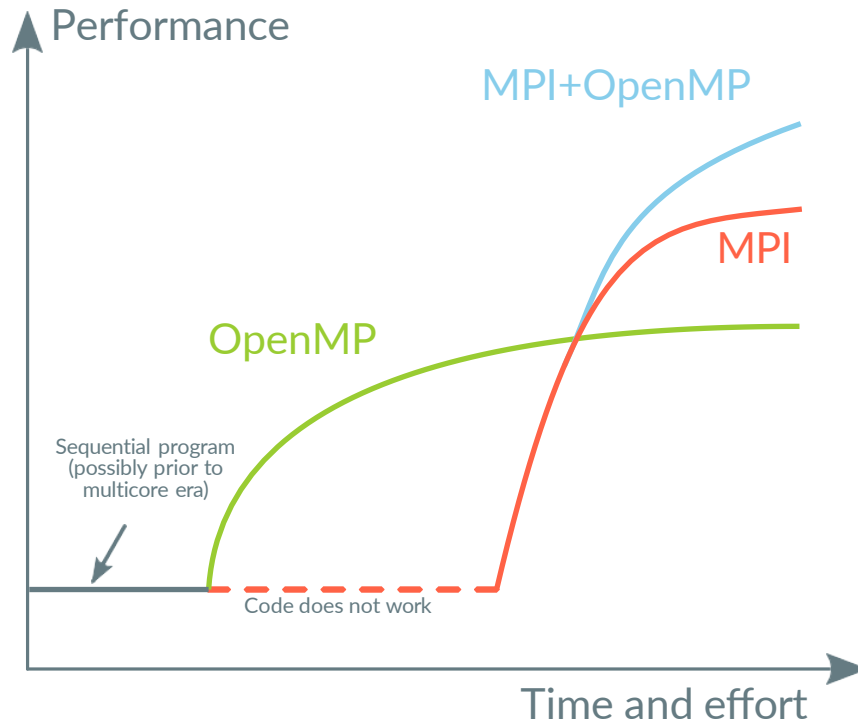
OpenMP: Open Multi-Processing



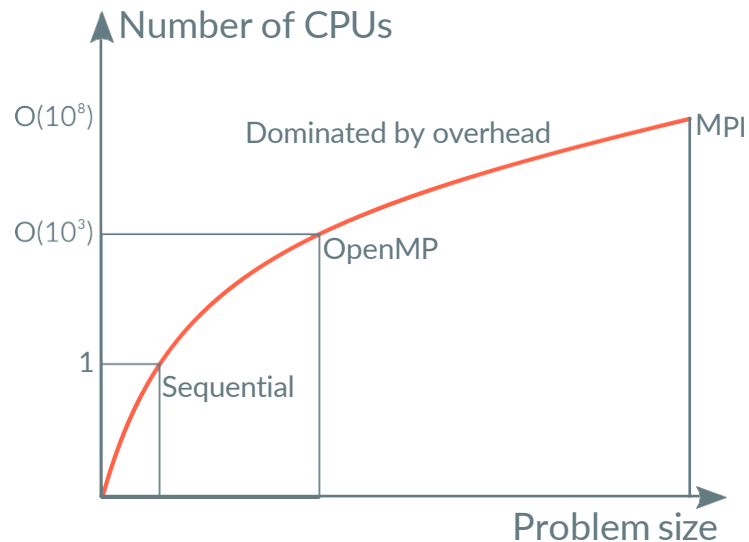
- Application Programming Interface (API) for explicit direct multi-threaded parallelism on shared memory architectures
 - Consists of 3 components:
 - Compiler directives
 - Runtime library calls
 - Environment variables
 - *A standard* widely used in industry and academic codes
-



- *Is not* required to check your code for data races, deadlocks or whether your code conforms with the standard
- Is not designed for distributed memory systems
- Is not designed for parallel I/O



- OpenMP support can be added very easily to *existing* sequential code via compiler directives. (Not necessarily the case for alternatives)
- Compiler simply ignores these directives if it does not recognize them. To tell the compiler about these pragmas, you must add the `-fopenmp` flag at compile time (for GCC).
- Other programming models are also based on compiler directives, e.g., OpenACC.



- OpenMP (shared memory) applications target nodes with roughly 16 to 128 cores. Upper bound may be ~1000 cores, although such systems are not the norm.
- Because the amount of memory on a node is limited, the target problem size is limited as well.
- For larger problems distributed memory is necessary (e.g., using MPI).
- Such large systems can become dominated by communication overhead. A method to alleviate this issue is to combine MPI and OpenMP into something hybrid

Easy to use with simple compiler directives and allows to parallelize existing sequential programs with minimum effort.

- Parallelization can be done incrementally. Work on one compute kernel, benchmark the effort and then proceed with other parallelization.
- Supports coarse-grained parallelism (e.g. via tasks) and, fine-grained parallelism (e.g. via locks and atomics).
- Widely used in industry and academia with large community support.
- Portability:
 - C, C++ and Fortran
 - Supports Unix/Linux as well as Windows

C++11 THREADS AND PTHREADS

C++11 threads example implementation

```
1 #include <thread>
2 #define NTHREADS 8
3
4 void do_work(const size_t tid)
5 {
6     // more local data
7     // do work
8 }
9
10 int main(void)
11 {
12     std::thread threads[NTHREADS];
13
14     // spawn threads
15     for (size_t t = 0; t < NTHREADS; ++t)
16         threads[t] = std::thread(do_work, t);
17     // join threads
18     for (size_t t = 0; t < NTHREADS; ++t)
19         threads[t].join();
20
21     return 0;
22 }
```

pthread example implementation:

```
1 #include <pthread.h>
2 #define NTHREADS 8
3
4 void *do_work(void *t)
5 {
6     /* thread ID */
7     long tid = (long)t;
8     /* more local data */
9     /* do work */
10    return NULL
11 }
12
13 int main(void)
14 {
15     pthread_t threads[NTHREADS];
16     void *status;
17     /* spawn threads */
18     for (long t = 0; t < NTHREADS; ++t)
19         pthread_create(&threads[t], NULL, do_work,
20                       (void *)t);
21     /* join threads */
22     for (int t = 0; t < NTHREADS; ++t)
23         pthread_join(threads[t], &status);
24     return 0;
25 }
```

Date	Version	Features Added
Oct 1997	Fortran 1.0	
Oct 1998	C/C++ 1.0	
Nov 1999	Fortran 1.1	
Nov 2000	Fortran 2.0	
Mar 2002	C/C++ 2.0	
May 2005	OpenMP 2.5	
May 2008	OpenMP 3.0	Con New
Jul 2011	OpenMP 3.1	final and mergeable clauses for task construct taskyield construct to allow user-defined task switching points min/max reductions in C/C++ OMP_PROC_BIND added
Jul 2013	OpenMP 4.0	Thread affinity and OMP_PLACES SIMD support added Tasking extensions Support for accelerators (e.g. GPUs)
Nov 2015	OpenMP 4.5	Support for Fortran 2003 firstprivate
Nov 2018	OpenMP 5.0	Extensions to memory model Initial support for C11, C++11, C++14, C++17 and Fortran 2008
Nov 2020	OpenMP 5.1	Full support for C11, C++11, C++14, C++17, C++20 and Fortran 2008 Initial support for Fortran 2018
Nov 2021	OpenMP 5.2	Reorganisation and changes made to improve the specification (better consistency and wording)

New OpenMP API 6.0 Examples Document

Provides programming examples of the OpenMP API, including new features found in the OpenMP 6.0 Specification.

[DOWNLOAD](#)

Large amount of the parallelization available from OpenMP compiler directives alone!
The main purpose of these directives is to fork/join parallel regions and provide synchronization primitives.

C/C++:

```
1 #pragma omp construct [clause [clause]...]  
2 _Pragma("omp construct [clause [clause]...]")
```

Fortran:

```
1 !$omp construct [clause [clause]...]  
2 c$omp construct [clause [clause]...]  
3 *$omp construct [clause [clause]...]
```

- Since these are all either compiler directives or comments in the case of Fortran, there are **no changes made on the code itself**.
- Most of these directives will execute code, in that case we call them a *construct*, e.g., `#pragma omp parallel` is a construct that executes a parallel region.
- Clauses are part of a directive and are *optional*. An example clause would be to specify how many threads you want to use for a parallel region: `#pragma omp parallel num_threads(2)`

```
1 #pragma omp directive-name [clause [[,]clause] ...] new-line
2     structured block
```

- After the directive is a *structured block*. Structured blocks are an executable statement (possibly compound between "{ }") and define a *single point of entry* and a *single point of exit*.

Structured block:

```
1 #pragma omp parallel
2 { // new scope: point of entry
3     int a = 0; // automatic variable
4     do_work(a); // do something in parallel
5 } // point of exit
```

There is a *single* point of entry at { and point of exit at }. It could also be a single statement without the "{ }".

Not a structured block:

```
1 #pragma omp parallel
2 { // new scope: point of entry
3     int a = 0; // automatic variable
4     do_work(a); // do something in parallel
5     if (a) goto out; // then go crazy!
6 } // point of exit
7 out: cout << "Skipped point of exit!\n";
```

The *goto* statement *jumps* out of the block, there will be *no single point of exit*!

Parallelizing existing code with OpenMP is frequently simple:

Sequential code:

```
1 int main(void)
2 {
3     int results[100];
4
5     for (int i = 0; i < 100; ++i) {
6         do_work(results[i]);
7     }
8
9     return 0;
10 }
```

```
1 $ g++ -o main main.cpp
```

Parallel code:

```
1 int main(void)
2 {
3     int results[100];
4
5     #pragma omp parallel for
6     for (int i = 0; i < 100; ++i) {
7         do_work(results[i]);
8     }
9
10    return 0;
11 }
```

```
1 $ g++ -fopenmp -o main main.cpp
```

However, keep in mind that as soon as you add the `#pragma omp parallel` line, you have signed the contract for *relaxed memory consistency* and responsibility for correct parallel code is now yours!

To compile OpenMP code you must tell the compiler to evaluate the `#pragma omp` pragmas. You do this by adding the `-fopenmp` flag to the compiler invocation

```
1 $ g++ -fopenmp [-o main] main.cpp # -o main creates an executable named main, otherwise it is a.out
2 $ gcc -fopenmp [-o main] main.c # same story for C
```

When you pass `-fopenmp` a macro named `_OPENMP` will be defined as well. This macro contains a string with the format `YYYYMM` which corresponds to the OpenMP release you are using. The macro is useful for *conditional compilation*:

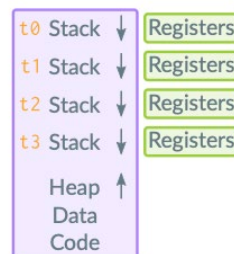
```
1 #include <iostream>
2
3 int main(void)
4 {
5     #ifdef _OPENMP
6         std::cout << "Compiled with -fopenmp "
7                     << "(OpenMP Standard: "
8                     << _OPENMP << ")\n";
9     #else
10        std::cout << "Compiled without -fopenmp\n";
11    #endif /* _OPENMP */
12    return 0;
13 }
```

Output:

```
1 $ make
2 g++ -O1 -fopenmp -o with_omp omp_conditional.cpp
3 g++ -O1 -o without_omp omp_conditional.cpp
4 $ ./with_omp
5 Compiled with -fopenmp (OpenMP Standard: 201511)
6 $ ./without_omp
7 Compiled without -fopenmp
```

thread

An execution entity with a stack and associated *thread-private* memory.



thread number

A number that the OpenMP implementation assigns to a thread. 0

Always the *primary* thread within a team of threads. The other child threads increment consecutively.

program order

An ordering of operations performed by *the same thread* as determined execution sequence of operators in the source code.

structured block

An executable statement, possibly compound, with a *single entry at the top* and a *single exit at the bottom* (i.e. a scope defined by "{}"), or an OpenMP construct.

directive

A C/C++ mechanism to specify OpenMP program behavior (e.g. compiler pragma or the `_Pragma()` macro).

conforming program

An OpenMP program that follows *all rules and restrictions* of the OpenMP specification.

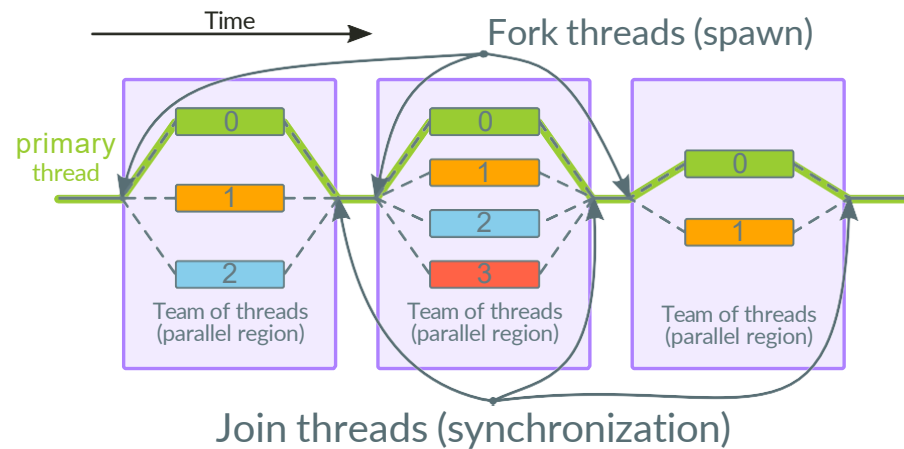
barrier

A point in the execution of a program encountered by a *team of threads* beyond which no thread in the team may execute until *all* threads in the team have reached the barrier (i.e., a *synchronization point*).

atomic operation

An atomic operation is an operation that will always be executed without any other thread being able to read or change the memory location that is read or changed during the operation.

Fork/join model:



- Forking and joining threads is done *transparently* in OpenMP using *directives*
- Your program *always* executes with at least one thread (primary thread)
- The primary thread spawns a *team* of threads at the beginning of a *parallel region*
- The primary thread always has ID 0

Parallel regions:

When a thread encounters a **parallel** construct, the thread creates a team of itself and zero or more additional threads. It becomes the *primary thread* of the new team.

A set of implicit tasks, one per thread, is generated. The code for each task is defined by the code inside the **parallel** construct.

parallel regions may be arbitrarily *nested* inside each other.

Synchronization constructs and library routines are available in the OpenMP API to coordinate tasks and data access in **parallel** regions.

- The OpenMP API provides a *relaxed* consistency shared memory model.
- Threads have a *temporary view* of the memory which includes registers, cache or other local storage.
- The temporary view allows threads to *cache* variables and therefore avoid going to main memory.

Types of variables inside parallel regions:

- A directive that accepts *data-sharing* attribute clauses determines *two kinds* of access variables used in the associated structured block: ***shared*** and ***private***.
- Each reference to a *shared variable* in the structured block becomes a reference to the original variable that exists outside of the structured block.
- For each *private variable* referenced in the structured block, a new version of the original variable is created in the thread private memory (of same type and size).

PARALLEL REGIONS

A program executes sequentially until it encounters a `parallel` directive.

The thread that *encounters* the directive first creates a *team of threads* (children) and it becomes the *primary thread* with number 0.

At the end of each `parallel` region, there is an *implicit barrier* that synchronizes the threads and joins them together again.

The *initial primary thread* then continues with sequential execution.

Directive to create parallel regions:

```
#pragma omp parallel [clause list] new-line  
    structured block
```

Control clauses:

```
1 if([parallel:] scalar-expression)  
2 num_threads(integer-expression)  
3 reduction(reduction-identifier: list)
```

Data clauses:

```
1 default(data-sharing-attribute)  
2 shared(list)  
3 private(list)  
4 firstprivate(list)
```

NUMBER OF THREADS IN PARALLEL REGIONS

The number of threads in a parallel region can be controlled by the following factors, in that order of precedence:

1. Setting the `num_threads` clause
2. Use of the `omp_set_num_threads()` library routine
3. Setting the `OMP_NUM_THREADS` environment variable at runtime
4. Fallback to *implementation default*. (This is typically the number of cores available on the machine but *do not rely on it!*)

Remember:

threads within parallel regions are *always* numbered starting from 0 (primary thread) to $n-1$ if there are n threads in the team. This also applies for *nested* parallel regions.

DYNAMIC AND STATIC MODES IN PARALLEL REGIONS

- **Dynamic mode** (the default)

The number of threads in different parallel regions in the same program **may differ**.

The specified number of threads is only an upper bound. The actual number of threads can be smaller to allow for optimization of resources at the OpenMP and OS level.

- **Static mode** (prefer this when running on HPC systems)

The number of threads used in parallel regions is static and exactly equal to what the programmer specifies by means of the `num_threads` clause, `omp_set_num_threads()` routine or `OMP_NUM_THREADS` environment variable.

You can enforce *static mode* by setting the following environment variable in your launch scripts:

```
1 OMP_DYNAMIC=false
```

USING THREADS TO GET WORK DONE

We now know how to spawn/fork a team of threads in OpenMP.
The next step is to accomplish parallelism with that tool.

Recall: there are two kinds of parallelism

1. Data-level parallelism
2. Task-level parallelism

In OpenMP there are two concepts: *worksharing* and *tasking*. The former (mostly) maps to data-level parallelism, the latter enables task-level parallelism

USING THREADS TO GET WORK DONE

In OpenMP there are two concepts: *worksharing* and *tasking*. The former (mostly) maps to data-level parallelism, the latter enables task-level parallelism

Worksharing: a way to distribute parallelizable work among a team of threads. Each thread then works on a *smaller subset* of the total workload in parallel.

Examples:

Array traversal in for-loops
Independent code sections (e.g. different independent functions)

Tasking: tasks can be used to exploit parallelism in *irregular* structures. A task consists of code, data and internal control variables.

Tasks are added to a queue and may execute immediately or get scheduled for later execution.

Tasks are cheap and help to avoid spawning threads dynamically.

Examples:

- Recursive functions
- List and tree traversal in parallel

OpenMP defines 3 worksharing constructs:

Loop construct: specifies that the iterations of *one or more* associated loops will be executed in parallel by the threads in the team. *This is worksharing construct is what you likely need most often.*

Sections construct: a *non-iterative* worksharing construct that contains a set of *structured blocks* that are to be distributed among and executed by the threads in a team.

Single construct: specifies that the associated structured block is executed by *only one* of the threads in the team (not necessarily the primary thread).

→ **Important to remember:** all worksharing constructs have an *implicit barrier* at the point of exit of the structured block, unless you use the *nowait* clause.

Parallelizing a loop "the raw way":

```
1  #include <omp.h>
2
3  int main(void)
4  {
5      const int N = 1000; // number of elements in array
6      double A[N];        // shared data array we want to work on in parallel
7
8      #pragma omp parallel
9      {
10         const int tid = omp_get_thread_num();
11         const int nthreads = omp_get_num_threads();
12         int npoints = N / nthreads; // work partition per thread
13         const int mystart = tid * npoints; // per thread array offset
14         if (tid == nthreads - 1) { // ensure to take care of remainders A
15             npoints = N - mystart;
16         }
17
18         // iterate over array A in parallel
19         for (int i = 0; i < npoints; ++i) {
20             A[mystart + i] = ... // do something here
21         }
22     }
23     return 0;
24 }
```

Tasks:

1. Get the worker information
2. Compute the work partition (similar to domain decomposition in MPI, you will see this later again)
3. Get the work done in parallel

Parallelizing a loop the OpenMP way:

```
1 int main(void)
2 {
3     const int N = 1000; // number of elements in array
4     double A[N];        // shared data array we want to work on in parallel 5
5
6     #pragma omp parallel
7     {
8         #pragma omp for
9         for (int i = 0; i < N; ++i) {
10             A[i] = ... // do something here
11         }
12     }
13     return 0;
14 }
```

- In line 8 we tell OpenMP our intention: *share* the work (i.e. parallelize) associated to the loop that follows.
- OpenMP takes care of the work partitioning (including remainders if N does not divide evenly among the number of threads).

Less code, fewer bugs, cleaner and more readable.

Parallelizing a loop the OpenMP way:

```
1 int main(void)
2 {
3     const int N = 1000; // number of elements in array
4     double A[N];        // shared data array we want to work on in parallel
5
6     #pragma omp parallel
7     {
8
9         for (int i = 0; i < N; ++i) {
10             A[i] = ... // do something here
11         }
12     }
13     return 0;
14 }
```

What's wrong here?

CANONICAL LOOP STRUCTURES

The `#pragma omp for` construct works with *canonical* loop-structures:

```
1         index++;
2         ++index;
3         index--;
4         <      --index;
5  for (index = start; index <= end; index += inc;   )
6         >=     index -= inc;
7         >      index = index + inc;
8         !=     index = inc + index;
9         index = index - inc;
```

SECTIONS CONSTRUCT

The sections construct is a non-iterative worksharing construct that contains a set of structured blocks that are to be distributed among and executed by the threads in a team. *Each structured block is executed once* by one of the threads in the team in the context of its implicit task.

```
1  int main(void)
2  {
3      #pragma omp parallel
4      {
5          #pragma omp sections // nowait [reduction(reduction-identifier: list)]
6          {
7              #pragma omp section
8                  x_calculation(); // independent computation
9              #pragma omp section
10                 y_calculation(); // independent computation
11             #pragma omp section
12                 z_calculation(); // independent computation
13             } // other threads in the team wait here. Use '#pragma omp sections
14               // nowait' to let them continue without synchronization
15         }
16         return 0;
17     }
```

SINGLE CONSTRUCT

The `single` construct specifies that the associated structured block is executed by *only one of the threads in the team* (not necessarily the primary thread!), in the context of its implicit task. The other threads in the team, which do not execute the block, wait at an *implicit barrier* at the end of a `single` region unless a `nowait` clause is specified.

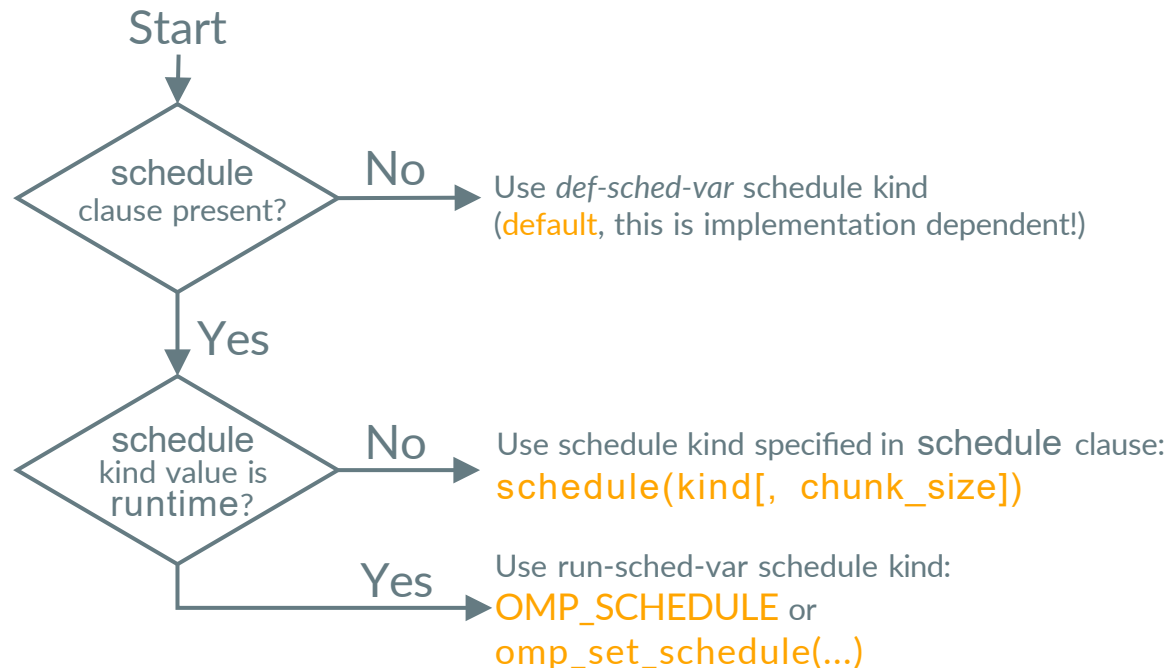
```
1 int main(void)
2 {
3     #pragma omp parallel
4     {
5         #pragma omp single // nowait
6         {
7             exchange_boundaries(); // irregularity here
8         } // other threads in the team wait here w/o nowait clause
9
10         // IFF this function does not depend on memory locations that need to
11         // be visible after exchange_boundaries() has been executed, then
12         // adding the 'nowait' clause to the single construct is more
13         // efficient!
14         do_many_things();
15     }
16     return 0;
17 }
```

LOOP SCHEDULING

You have control over how work shall be distributed among threads when you parallelize for-loops. We *do care* about this because it allows to adjust the *parallel granularity* and control *load balancing*.

```
1 #pragma omp for schedule(kind[, chunk_size]) new-line
2   for-nest
```

How does OpenMP determine the schedule:



The *def-sched-var* is an internal control variable and *depends on the implementation* (gcc may do things differently by default than clang does)

If you rely on a specific schedule, you must always specify it in your code.

LOOP SCHEDULING KIND

`static[, chunk]`

Loop iterations are divided into segments of size `chunk` and distributed cyclically to the threads in the team

If `chunk` is not specified, it *might* be similar in size to N/n_{threads} (implementation dependent)

`dynamic[, chunk]`

Loop iterations are divided into segments of size `chunk`

An *idle thread* is *dynamically* assigned the next available chunk. The `chunk` size defaults to 1 if not specified

`guided[, chunk]`

Similar to `dynamic` but the `chunk` size *decreases exponentially*

`chunk` specifies the *minimum* chunk size. Defaults to 1 if not specified

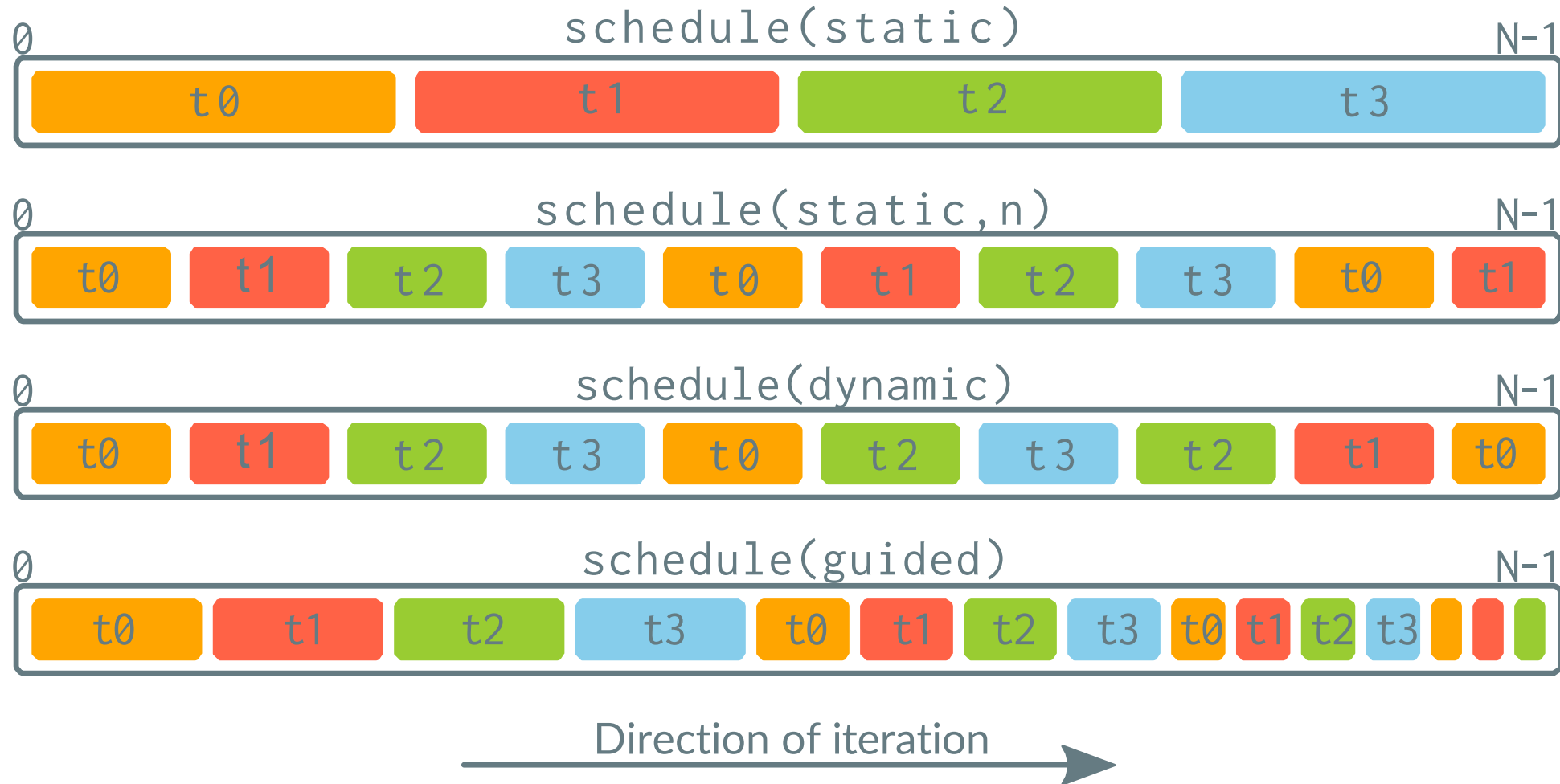
`runtime`

Decide at runtime depending on the value of the `OMP_SCHEDULE` environment variable or the `omp_set_schedule()` library call at some point in your code

`auto`

The decision is delegated to the compiler and/or the OpenMP runtime

LOOP SCHEDULING KIND VISUALLY



NESTED LOOPS

You can parallelize nested loops using OpenMP:

```
1 int main(void)
2 {
3     const int N = 1000;
4     double A[N][N]; // 2D array
5
6     #pragma omp parallel
7     {
8         #pragma omp for
9         for (int i = 0; i < N; ++i) // this loop is run in parallel
10            for (int j = 0; j < N; ++j) // each thread runs this loop from [0,N)
11                A[i][j] = ... // do something here
12    }
13    return 0;
14 }
```

The "#pragma omp for" only applies to the for-loop that *immediately* follows it

Each thread executes the innermost loop in full

Important general remark for parallel loops: recall that a structured block must have a single point of exit. You *cannot* have a break statement in the outermost loop (the one to which "#pragma omp for" applies)!

COLLAPSING NESTED LOOPS

To increase the amount of work you parallelize, nested loops can be *collapsed* under certain conditions:

```
1 #pragma omp for collapse(n) new-line  
2   for-nest
```

- n must be a constant positive integer expression and denotes the number of loops to collapse
- You can only collapse *perfectly nested* loops
- **Perfectly nested:** the loop body of the outer loop can consist entirely of the inner loop
- The loops you collapse **must not** modify any of the loop iteration indices! (e.g. i and j)
- Benefits of collapsing loops:
 - Can achieve better resource utilization due to larger iteration space
 - Better load-balancing behavior of the parallelized loop

COLLAPSING NESTED LOOPS

Example: large difference in dimensions of array

```
1 int main(void)
2 {
3     double A[5][10000]; // 2D array
4
5     #pragma omp parallel
6     {
7         #pragma omp for collapse(2)
8         for (int i = 0; i < 5; ++i)
9             for (int j = 0; j < 10000; ++j)
10                A[i][j] = ... // do something
11     }
12     return 0;
13 }
```

Without the collapse(2) clause, *only 5 threads* work on the loop in parallel, the others are idle

Example: GEMV (matrix-vector product)

```
1 int main(void)
2 {
3     const int N = 1000;
4     double x[N], y[N]; // Vectors 5
5
6     double A[N][N];      // Matrix
7     // initialize data...
8     // parallel matrix vector product
9     #pragma omp parallel
10    {
11        #pragma omp for collapse(2)
12        for (int i = 0; i < N; ++i) {
13            y[i] = 0.0;
14            for (int j = 0; j < N; ++j)
15                y[i] += A[i][j] * x[j];
16        }
17    }
18    return 0;
19 }
```

Can you collapse the loops in line 12 and line 14?

REDUCTION

A reduction is a class of parallel algorithms that transform $O(N)$ inputs to $O(1)$ output by means of an associative binary operator $\oplus$.

Examples include:

- finding a global minimum or maximum
- summation of values or sum of squares
- product sums such as an inner product ubiquitous in linear algebra
- logical operations

```
1 #pragma omp for reduction(reduction-identifier: list) [other clauses] new-line
2     for-nest
```

Reduction identifier: this is the binary operator $\oplus$. This operator can be one of the following built-in operators

```
1 +, -, *, &, |, ^, &&, ||, min and max
```

- list is a comma separated list of reduction variables
- Can also create user-defined reduction operators
- The reduction clause can appear multiple times but not for the same reduction variable

Initial values: OpenMP creates *thread private* variables for the variables in the list. They are initialized as follows

-:	0
*:	1
&:	~0
:	0
^:	0

:	0
min:	Maximal representable number in the reduction list item type
max:	Minimal representable number in the reduction list item type

REDUCTION

Example: sum the elements in an array in parallel

```
1  int main(void)
2  {
3      double a[1000];
4      // initialize data... 5
5
6      double sum = 0.0; // sum the elements in here, must be initialized properly
7      #pragma omp parallel
8      {
9          #pragma omp for reduction(+: sum)
10         for (int i = 0; i < 1000; ++i)
11             sum += a[i];
12     }
13     return 0;
14 }
```

line 6

The variable `sum` is initialized to 0 and is stored in shared main memory.

line 9

OpenMP creates *private copies* of `sum` for each thread and initializes them to 0 (because of the `+` reduction operator).

line 11

Each thread sums up its array partition into its own private copy of `sum` (no race condition!). The result depends on the data the thread works on and will be different among threads.

line 12

OpenMP *reduces* the partial sums into the shared `sum` variable. It will *add* these partial sums to whatever the value of `sum` was before the reduction.

REDUCTION

Example: sum the elements in an array in parallel

```
1 int main(void)
2 {
3     double a[1000];
4     // initialize data...
5
6     double sum = 0.0; // sum the elements in here, must be initialized properly
7     #pragma omp parallel
8     {
9         #pragma omp for reduction(+: sum)
10         for (int i = 0; i < 1000; ++i)
11             sum += a[i];
12         process_result(sum);
13     }
14     return 0;
15 }
```

Is this code correct?

NEXT TIME

OpenMP examples